The Agreement Trap

When ARC-AGI-2 Solvers Agree on the Wrong Grid

Xinyi Fang

Abstract. Magpie, our ARC-AGI-2 solver, often finds the right answer and then votes it down: across pooled runs, about a third of the correct grids it generates lose the vote to a wrong one. The reason is that transformed copies of one adapted model are not independent voters. In 19 of 20 audited misses the winning wrong grid is produced by every run, and on average it outvotes the correct grid 13.1 to 3.7. Even in the most unanimous fifth of inputs, Magpie is right only half the time, and no reranking of its own signals closes the gap. The trap also predicts what helps. More of the same model adds nothing: halving its views costs 0.25 points, and extra augmentation adds none. Splitting the same budget between two independently trained adapters adds 1.7 points ($t \approx 7$). A reasoning model that misses different puzzles is the best outside voter, until it is shown Magpie's candidates; then its accuracy where Magpie is wrong falls from 31% to 16%. Under the shared 12-hour clock, both solvers crowd onto the same cheap puzzles, and none of our local gains reached the leaderboard, where one unchanged notebook scores anywhere from 27.2% to 32.6%.

1 Introduction

An ARC-AGI-2 puzzle shows a few demonstration pairs of colored grids and asks for the output of a new test input. A solver may submit two guesses, and a guess counts only if every cell is right [2, 4]. The benchmark measures how efficiently a system acquires a new skill, not how much it already knows [1]. Every ARC-AGI-2 task has been solved by at least two people in no more than two attempts [4], yet the best Kaggle entry of 2025 scored 24% on the private set under the competition's compute limit [3]. Closing that gap takes more than generating answers; a solver also has to choose among them. That is where this paper started: Magpie kept finding answers it did not submit.

Magpie, our submission, follows the recipe of recent top ARC Prize entries [6, 8]: train a small adapter on each puzzle, ask it for answers on rotated, reflected and recolored copies of the test input, and let the copies vote. The recipe finds far more answers than it submits. Pooling six runs, Magpie generates the correct grid on tasks worth about 57 of 120 points and submits it on tasks worth 36.67. About twenty points of correct answers are generated and then voted down.

A vote is evidence only when the voters make independent mistakes, and copies of one adapted model do not. When the model misreads a puzzle, it misreads every rotation of it the same way, so a wrong grid collects votes exactly as fast as a right one would. Agreement then measures how consistent the model is, not whether it is right. Test-time training builds this in: the adapter is fitted to give consistent answers under the very transformations that later cast the votes. We call the result the *agreement trap*.

Two guesses exist for puzzles like Figure 1, where the evidence fits several answers and no vote could choose between them. The misses we study are the opposite. The evidence decides, the correct grid is generated, and it loses the vote. Following those lost answers through Magpie and its partner solver led to four findings.

Agreement is a weak certificate (Section 4). In 19 of 20 audited misses, the wrong grid that wins is reproduced by every run. On the inputs where Magpie's vote is most unanimous, its answer is right only 51% of the time, and no reranking of its own signals brings the lost answers back.

More views add nothing; a second model adds points (Section 5). Halving the transformed copies costs 0.25 points. Spending the same budget on two independently trained adapters adds 1.7. Pooled

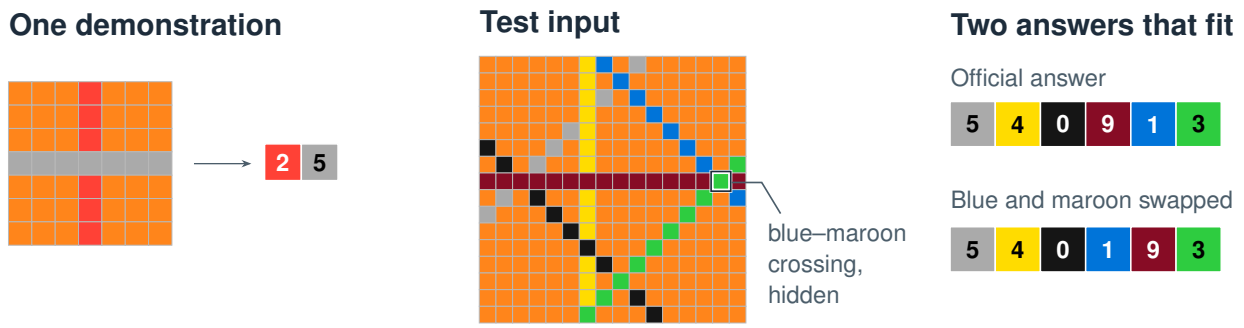


Figure 1. A puzzle that no vote can settle. Each output lists the input's line colors from the bottom layer to the top. In the demonstration, gray lies over red, so the answer is red, then gray. In the test input, the only point where the blue (1) and maroon (9) lines cross is covered by green, so nothing shows which of the two is higher. Of the 720 possible layer orders, exactly four reproduce the test input (Appendix A). The official answer is the first strip; swapping blue and maroon gives the second, which fits the evidence equally well. Public training task `22425bda` [5].

runs help only when they were trained differently.

An outside voter must stay outside (Section 6). A reasoning model that fails on different puzzles is the most valuable second opinion Magpie has, until it sees Magpie's candidates. Then its accuracy where Magpie is wrong falls from 31% to 16%. Its own stopping behavior is a better confidence signal than any vote: answers it finishes on its own are right 66% of the time, answers cut off at the token limit 6%.

The clock decides (Section 7). Both solvers earn two thirds of their score in the first half of their budget, on the cheapest puzzles, so one shared 12-hour clock sends them to the same puzzles. None of our local gains reached the leaderboard, where one unchanged notebook scores anywhere from 27.22% to 32.64%.

Magpie's reference configuration averages 29.51% on the public leaderboard over 15 reruns, with a best of 32.64%.

2 Prior work

Test-time training and augmentation voting. The strongest neural ARC solvers adapt a model to each puzzle at test time and aggregate its predictions over transformed copies of the input. MindsAI introduced test-time fine-tuning together with AIRV: augment, infer, reverse the augmentation, and vote [11]. Akyürek et al. showed that per-puzzle low-rank adaptation multiplies accuracy several-fold [7]. The ARChitects scored candidates by their probability under many augmented views, a product of experts [8]. NVARC won ARC Prize 2025 by combining such adaptation with large-scale synthetic training data [3, 6]. Magpie inherits this stack, and like Magpie, all of these treat agreement across transformed copies as evidence. We show when it is not: the copies share the adapted model's mistakes, so more of them add nothing, and the evidence for the right grid has to come from a model trained differently.

Voting and correlated errors. Majority voting over sampled reasoning paths improves language-model accuracy [12]. The classical analysis of ensembles explains when: an ensemble's error equals its members' average error minus their disagreement, so the gain vanishes when their mistakes coincide [13]. Across hundreds of language models, Kim et al. found errors strongly correlated: on one benchmark, two models that are both wrong give the same wrong answer 60% of the time, and stronger models agree more [14]. We find the same failure inside a single solver, and show that two independently trained adapters recover the disagreement that more copies cannot.

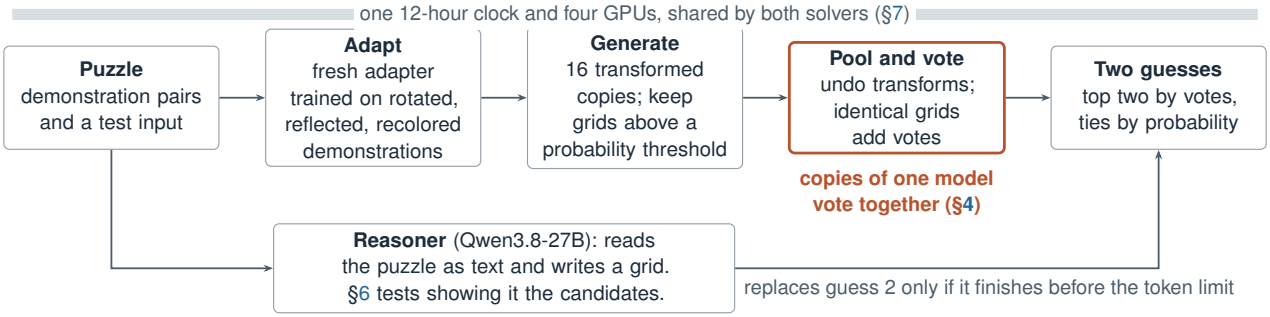


Figure 2. Magpie at a glance. The grid solver's candidates from all transformed copies are pooled and ranked by votes. The reasoner works on the same puzzle in whatever time is left and may replace only the second guess. Orange marks where agreement can mislead.

Combining solvers. Li et al. found that program induction and direct transduction solve different ARC tasks, and that combining them beats either alone [10]. Separately, language models are easily anchored by answers placed in their prompt, even when told to disregard them [15]. We connect the two observations: a second solver is valuable because it fails differently, and showing it the first solver's candidates costs it a quarter to a half of its value on exactly the inputs where it was needed.

Program search. Program-synthesis approaches sample or evolve thousands of candidate programs and keep those that reproduce every demonstration [10, 16]. Our pilots show that fitting the demonstrations is itself a form of agreement, and that search can inflate it without solving more test inputs.

Reasoning length and speculative decoding. Hassid et al. report that, for a given question, shorter thinking chains are more often correct than longer ones [17]. We find the same signal on ARC and use it as a hard rule for the second guess. Our faster decoder is a form of speculative decoding [18], with the draft taken from another transformed copy's answer instead of from a smaller model.

3 How Magpie produces two guesses

The grid solver. We use NVARC's public Qwen3-4B checkpoint and its test-time training recipe [6–8]. For each puzzle, a fresh low-rank adapter (a small set of extra trainable weights) is fine-tuned on the demonstrations and their rotated, reflected and recolored copies. The adapted model then reads 16 transformed copies of the test input, and a depth-first search over output tokens returns every grid above a probability threshold. Mapped back to the original orientation, identical grids count as votes. The two grids with the most votes are submitted, with ties broken by the model's probability of each grid combined over eight transformed copies.

The reasoner and the hybrid. Qwen3.8-27B [9] reads the puzzle as text, reasons in words and writes a grid. In the hybrid both solvers share four GPUs. The grid solver runs first and the reasoner gets the rest of the window; an answer the reasoner finishes before its token limit replaces Magpie's second guess.

Scores. The *local score* uses the 120 public evaluation tasks (172 test inputs). Exact matches are averaged within each task and summed, so the maximum is 120; Section 6 reports percentages of test inputs instead. The *leaderboard score* comes from one 12-hour offline Kaggle run over 240 hidden tasks on four L4 GPUs. The public and hidden sets are calibrated to the same difficulty within one percentage point [4], and for the grid solver the two scales agree: one adapter averages 36.02 locally (30.0%) and the reference averages 29.51% on the leaderboard.

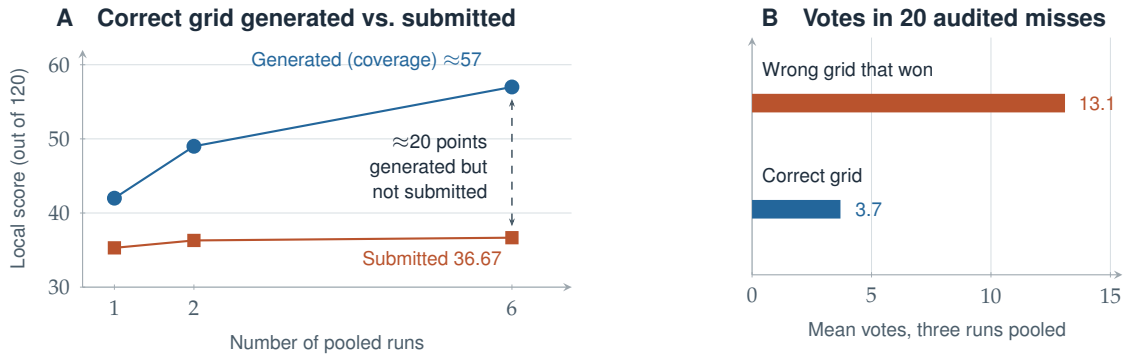


Figure 3. Magpie generates far more correct answers than it submits, and the wrong answers win by a landslide. (A) One and two runs are averaged over runs drawn from six seeds; six runs are pooled together. (B) Twenty audited tasks where the pool held the correct grid but the submission missed it, with mean votes over three pooled runs. The winning wrong grid appeared in all three runs on 19 of the 20.

4 Agreement is a weak certificate

We separate two quantities on the local score. *Coverage* counts tasks whose correct grid appears anywhere among the pooled candidates; it is what a perfect ranking could submit. The *submitted score* counts tasks where one of the two submitted grids is correct.

Pooling buys candidates, not points. Pooling more runs of the same configuration, each with its own random seed, raised coverage from about 42 to 57 points while the submitted score went from 35.3 to 36.67 (Figure 3A). Of the roughly 15 points of correct answers the extra runs added, the ranking turned 1.4 into submissions. A single run already generates 6 to 8 points of correct answers that it does not submit; six pooled runs leave 20 on the table. The runs disagree about which correct grids they generate, and agree about which wrong grid they vote for.

The wrong winners are reproducible. We examined 20 tasks where three pooled runs held the correct grid and the submission still missed it. The correct grid appeared in 2.15 of the three runs on average and drew 3.7 votes. The grid that beat it appeared in all three runs on 19 of the 20 tasks and drew 13.1 votes (Figure 3B), and the model's own probability favored it on 18. Magpie is not guessing differently each time and occasionally guessing wrong. It is wrong the same way in every run and every orientation, and that is exactly what makes its vote confident.

Unanimity is not certainty. Over ten runs on all 172 test inputs, we sorted inputs by the share of votes their winning grid received. In the least unanimous fifth, Magpie's answer was right 14% of the time. In the most unanimous fifth it was right 51% of the time, and even where nine votes in ten agreed, it was right only about half the time. Agreement separates hopeless inputs from coin flips. It does not certify an answer.

Nothing inside the model breaks the tie. We tried six families of selection rules over the stored signals: vote counts, probabilities under each transformed copy, stability across runs, vote-gap rules, hedging between two rankers, and a public portfolio ranker. The best rule gained less than half a point on average when re-ranking the same candidates across 13 runs. A logistic regression over 16 features, trained on some puzzles and tested on others, changed the score by -0.03 over 15 runs and by -0.01 on two-adapter pools; its learned weights reproduce the inherited rule. The correct grids that lose are not mis-weighted. Every signal the model has scores them lower, so the evidence that would rescue them has to come from outside the model.

The same trap in program search. Fitting the demonstrations is agreement too. A 7B coding model found demonstration-perfect programs for 14 of its first 35 public tasks and solved none of them. With a 27B model on 57 held-out tasks, evolution raised the number of demonstration-perfect tasks

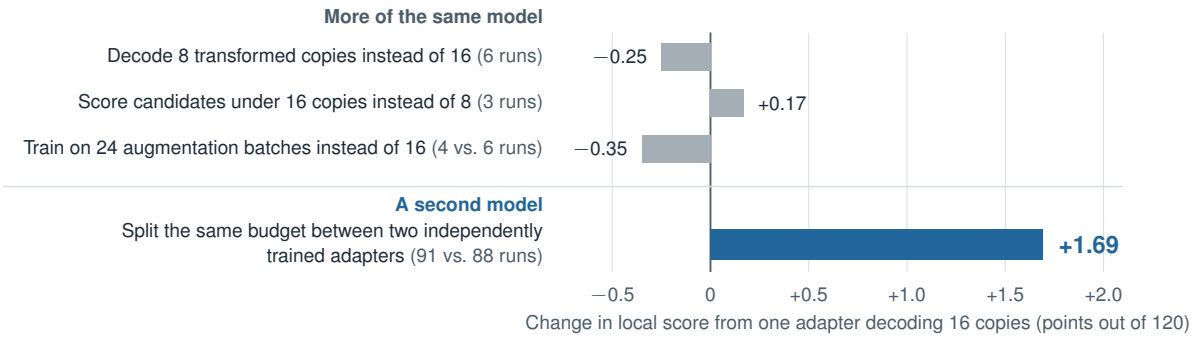


Figure 4. More copies of one model are worth nothing; a second model is worth 1.7 points. Gray rows add or remove copies of the same adapted model. The blue row keeps the training and decoding budget fixed and splits it between two independently initialized adapters, about seven standard errors above a single adapter.

from 22 to 34 while correct test answers stayed at 20 (21 and 19 for plain sampling). Search can manufacture agreement with the demonstrations without getting any closer to the answer.

5 More views add nothing; a second model adds points

If correlated voters cause the trap, then more copies of the same model should not help, and a model trained differently should. Both predictions hold (Figure 4).

More of the same model. Decoding 8 transformed copies instead of 16 costs 0.25 points over six runs. Scoring candidates under 16 copies instead of 8 changes almost nothing (+0.5, 0 and 0 in three runs). Training the adapter on 50% more augmented data scores 0.35 points lower (four runs against six). In an earlier build, 4, 8, 16 and 32 batches of augmented training data all landed in the same band. Extra copies of one model are extra copies of its opinion.

A second model, at the same cost. We replaced the single adapter with two adapters initialized independently, each trained on half the augmented data and each decoding half the copies, and pooled their candidates, for about 7% more time. Over repeated runs the local score rose from 36.02 (SD 1.56, 88 runs) to 37.71 (SD 1.48, 91 runs): 1.69 points, about seven standard errors. The gain comes from pooling the two adapters' candidates, and scoring each candidate under both adapters adds nothing measurable. A third adapter adds only 0.2 points for 45% more time. Two independent opinions capture most of the disagreement there is to buy.

Pooled runs help only when they disagree. An earlier build shows the same law across whole runs. Pooling two runs of the same configuration gained 0 to 1 point. Pooling a run trained on 8 augmentation batches with one trained on 16 gained 2 to 4 points in all nine pairs we tried, with a best of 42.33. Pooling all eight available runs scored only 38.67, although together they held the correct grid on tasks worth 56.33: each added run of an already represented configuration added more votes for the same mistakes.

Faster search bought nothing. We made the search 2.95× faster by reusing the grid that one transformed copy has already produced as a draft for the next, so the model checks several cells per forward pass instead of writing them one at a time (17,316 to 5,865 seconds over 120 tasks). The faster build averaged 28.96% on the leaderboard over two reruns, inside the reference range. A faster search buys more candidates of the same kind, and those are not what the ranking lacks.

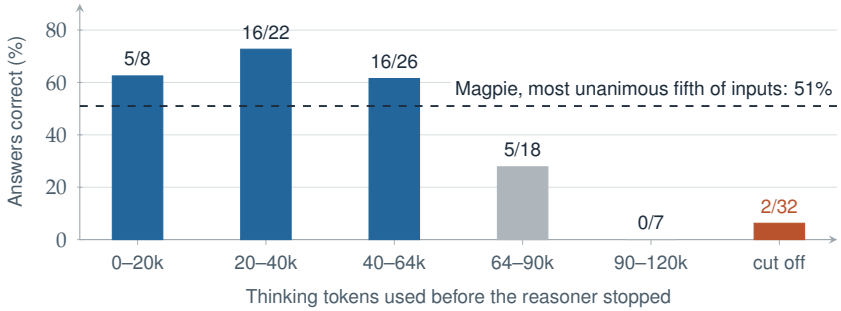


Figure 5. The reasoner knows when it knows. Accuracy of the reasoner's answers by how many thinking tokens it used before stopping on its own: blue within the 64,000-token limit used in competition, gray beyond it. The orange bar shows answers cut off at the limit and forced out. Labels give correct/total. The dashed line marks Magpie's accuracy on the fifth of inputs where its vote was most unanimous.

6 An outside voter must stay outside

The reasoner fails on different puzzles. On the public tasks the grid solver answered 47–55 test inputs correctly across runs. The reasoner answered 44, having reached only 113 of the 172 inputs in time, and only 15–18 correct answers were shared. Together they solve 74–81 inputs, tasks worth 53–57 points against 34–42 for the grid solver alone [cf. 10]. This is the independent voter Section 4 asked for.

It knows when it knows. Magpie's votes are a weak certificate; the reasoner's own stopping behavior is a strong one (Figure 5). Answers it finished on its own within 64,000 thinking tokens were right 37 times in 56 (66%). Beyond 64,000 tokens accuracy collapses: 5 of 18 up to 90,000 tokens and none of 7 beyond. Answers cut off at the limit and forced out were right 2 times in 32 (6%). This signal pays exactly where Magpie is weakest. Its second guess supplies about a quarter of its solves at roughly 10% precision, so in the hybrid only an answer the reasoner finished on its own may replace it. The signal also says how to spend the budget: one long attempt beats two short ones, so the reasoner samples again only when its first attempt is cut off.

Shown the candidates, it stops being outside. We reran the reasoner on the same inputs with up to five distinct candidates from Magpie in the prompt, along with a note that all of them might be wrong and that it should solve the puzzle itself. The candidates came from two Magpie runs (Figure 6). Where Magpie was right, the reasoner improved, from 42% to 50% and from 45% to 58%. Where Magpie was wrong, it fell from 31% to 16% and from 31% to 24%. Only the second group can earn new credit, since an input Magpie already solves gains nothing when the reasoner solves it too. We call accuracy on that group *rescue accuracy*, and by it the candidates cost the reasoner between a quarter and half of its value. It thought just as long as before and simply ended closer to Magpie. It agreed more where agreement was free, and it abandoned its own answer where its own answer was the point.

Choosing among candidates is not an outside opinion either. We asked a 27B model, without extended reasoning, to pick the correct grid from Magpie's four best distinct candidates. It picked right 18 times in 44 (41%, against 25% by chance). When the correct grid was already Magpie's first choice, the judge kept it only 14 times in 29. Used to fill the second guess, it would have rescued one answer and broken five. An outside opinion is worth having only when it is formed from the puzzle, not from Magpie's shortlist.

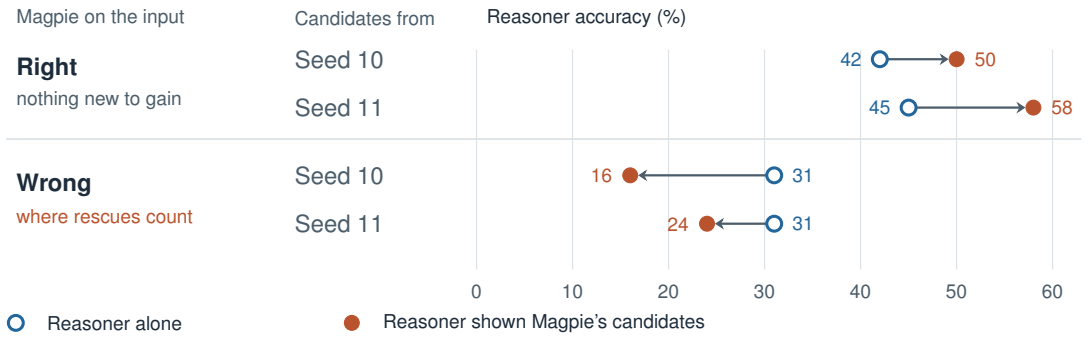


Figure 6. Shown Magpie's candidates, the reasoner gains where it is not needed and loses where it is. Each arrow joins the reasoner's accuracy on the same inputs without and with up to five of Magpie's candidates in its prompt. The candidates come from two Magpie runs (seeds 10 and 11, over 132 and 105 inputs).

7 The clock decides

Two solvers chase the same cheap puzzles. Both solvers front-load their score. The grid solver earns 69% of its local score in the first half of its time and 80% in the first 60%; the reasoner earns 68% in the first half of its tokens. On Kaggle's L4s the grid solver's first pass needs a median of about 760 seconds per task, while the leaderboard run allows about 700 (240 tasks on four GPUs in under 12 hours). The reasoner is no better off: on the same GPUs it tops out near 500 tokens per second, under one full-length attempt per test input. Given 60% of the window in the hybrid, it covers about 57% of its workload and never reaches roughly the most expensive 30% of the tasks. The reasoner, also working cheapest-first in the remaining time, spends that time on puzzles the grid solver already had. Replaying our local runs under this split puts the hybrid slightly below the grid solver alone (35.70 against 36.32 points). On the leaderboard it scored 29.03% against 29.51%. The two solvers make different mistakes, but the shared clock never let them work on different puzzles.

The obvious fix finds the reasoner's limit. Sending the reasoner first to the inputs the grid solver never reached, then to those it was least confident about, scored 26.81%; this build also halved the grid solver's training to make room. The same replay attributes about 1.4 points of that score to the reasoner, against about 5.4 when it took the small puzzles first. On the hidden set, the large puzzles the grid solver could not reach were beyond the reasoner too. Giving the grid solver 80% of the window instead scored 26.39% (Table 1).

Minutes are points. Time is also the likeliest reason the two-adapter gain never reached the leaderboard. Locally that gain is worth about 1.4 percentage points; on the leaderboard the two-adapter build averaged 28.40% over four reruns, against 28.96% for the matched one-adapter build. It needs about 7% more time per task in a run whose first pass already barely fits. Taking ten more minutes from the main phase for a longer finishing margin lowered it again, to 27.02% over two reruns.

A single leaderboard score is one draw. The unchanged reference notebook, rerun 15 times, scored between 27.22% and 32.64% (SD 1.39). Two public notebooks, advertised at 33.89% and at 33.89–35.97%, scored 28.06% and 27.36% when we resubmitted them unchanged. Confirming a 1.4-point difference at two standard errors takes about eight reruns per system, so one or two submissions cannot rank systems that differ by less than a few points.

8 Conclusion

Agreement among copies of one model is not evidence. Magpie's wrong winners are reproduced by every run in 19 of 20 audited misses, and its most unanimous answers are right half the time.

System (number of leaderboard runs)	Score (%)
Reference grid solver (15)	29.51 (27.22–32.64)
Faster search, one adapter (2)	28.96
Two adapters, 15-minute finishing margin (4)	28.40
Two adapters, 25-minute finishing margin (2)	27.02
Reasoner alone (1)	25.28
Hybrid: grid solver 60%, reasoner takes small puzzles first (1)	29.03
Hybrid: grid solver 60% at half training, reasoner takes unreached inputs first (1)	26.81
Hybrid: grid solver 80%, reasoner takes small puzzles first (1)	26.39
Public notebook advertised at 33.89, resubmitted unchanged (1)	28.06
Public notebook advertised at 33.89–35.97, resubmitted unchanged (1)	27.36

Table 1. Nothing beat the unchanged reference, and advertised scores did not replicate. Rows with several runs give the mean; the reference row also gives the range of its 15 reruns. Percentages are for the leaderboard's 120 tasks.

More runs fill the candidate pool, not the answer sheet.

Buy disagreement, not repetition. Halving the views costs a quarter of a point; a second, independently trained adapter at the same cost adds 1.7. Pool runs only when they were trained differently.

Keep the outside voter outside, and trust how it stops. A partner shown the candidates loses up to half its rescue value, and a judge choosing among them breaks five answers for every one it saves. A reasoner's self-finished answers, right 66% of the time, say more than any vote count.

Under a fixed clock, minutes are points. Two solvers that make different mistakes still chase the same cheap puzzles. A single leaderboard score moves by five points from one rerun to the next.

Acknowledgments

We thank the ARC Prize Foundation for making the ARC-AGI-2 benchmark and its public tasks available. This work used computing resources of the Department of Computer Science and the High Performance Computing Cluster at Brandeis University.

A Implementation and reproduction details

Builds. Unless noted, results use the reference grid solver: 16 batches of augmented demonstrations (128 training steps, learning rate 5×10^{-5}) and 16 decoded copies. It starts with cheaper tasks, reserves the last 15 minutes for finishing, and uses leftover time for cheaper repeat passes. The configuration-pooling comparison and the augmentation curve in Section 5 and the outside judge in Section 6 come from an earlier build with the same checkpoint, on the same 120 public tasks. The program-synthesis pilots used a 7B coding model and Qwen3.6-27B.

Reasoner. The competition notebook runs Qwen3.8-27B in FP8 across four L4 GPUs with a 64,000-token thinking limit. Local reasoner measurements ran in bf16 on L40 GPUs with a 120,000-token limit. The two candidate lists in Figure 6 came from grid-solver seeds 10 (132 inputs, H100) and 11 (105 inputs, L40).

Reproducing Figure 1. In zero-based row and column coordinates (r, c), the six straight lines in the test input occupy $r + c = 9$ (gray), $c = 6$ (yellow), $c = r - 5$ (black), $r = 7$ (maroon), $c = r + 7$ (blue)

and $r + c = 21$ (green). Painting them in all $6! = 720$ layer orders on the orange background, exactly four orders reproduce the test input. Figure 1 shows two, including the official answer.

Configuration	Notebook / version	Submission ID(s)
Reference grid solver, best of 15 reruns	`magpie` / 28	56623279 (32.64%)
Faster search, one adapter	`magpie` / 33	56809077, 56836580
Two adapters, 15-minute margin	`magpie` / 36–38	56864684, 56894393, 56922534, 56925497
Two adapters, 25-minute margin	`magpie` / 39–40	56974757, 56979049
Qwen3.8-27B reasoner alone	`magpie-q38` / 7	57377849
Three hybrids, in Table 1 order	`magpie-h` / 1, 2, 4	57413215, 57456825, 57640698
Public notebooks, resubmitted unchanged		55228230, 57500979

Table 2. Competition entries in the `squaree` namespace. Notebooks: Magpie, Magpie-Q38 and Magpie-H.

References

[1] F. Chollet. On the Measure of Intelligence. 2019.

[2] F. Chollet, M. Knoop, G. Kamradt, B. Landers, and H. Pinkard. ARC-AGI-2: A New Challenge for Frontier AI Reasoning Systems. 2025.

[3] F. Chollet, M. Knoop, G. Kamradt, and B. Landers. ARC Prize 2025: Technical Report. 2026.

[4] ARC Prize Foundation. ARC Prize Guide.

[5] ARC Prize Foundation. Public training task 22425bda. ARC-AGI-2 repository.

[6] I. Sorokin and J.-F. Puget. NVARC solution to ARC-AGI-2. 2025.

[7] E. Akyürek et al. The Surprising Effectiveness of Test-Time Training for Few-Shot Learning. ICML, 2025.

[8] D. Franzen, J. Disselhoff, and D. Hartmann. Product of Experts with LLMs: Boosting Performance on ARC Is a Matter of Perspective. ICML, 2025.

[9] Qwen Team. Qwen3.8-27B model card. 2026.

[10] W.-D. Li et al. Combining Induction and Transduction for Abstract Reasoning. 2024.

[11] J. Cole and M. Osman. Don't Throw the Baby Out with the Bathwater: How and Why Deep Learning for ARC. 2025.

[12] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou. Self-Consistency Improves Chain of Thought Reasoning in Language Models. ICLR, 2023.

[13] A. Krogh and J. Vedelsby. Neural Network Ensembles, Cross Validation, and Active Learning. NIPS, 1994.

[14] E. Kim, A. Garg, K. Peng, and N. Garg. Correlated Errors in Large Language Models. ICML, 2025.

[15] J. Lou and Y. Sun. Anchoring Bias in Large Language Models: An Experimental Study. 2024.

[16] R. Greenblatt. Getting 50% (SoTA) on ARC-AGI with GPT-4o. Redwood Research, 2024.

[17] M. Hassid, G. Synnaeve, Y. Adi, and R. Schwartz. Don't Overthink It: Preferring Shorter Thinking Chains for Improved LLM Reasoning. 2025.

[18] Y. Leviathan, M. Kalman, and Y. Matias. Fast Inference from Transformers via Speculative Decoding. ICML, 2023.